# One rescue. Functions, all the way down.

Lambda World · Jiri Knesl · 40 minutes

Timed speaking draft. Times include demos and deliberate pauses. Rehearse aloud on the venue machine; use each chapter’s cut and recovery instructions when needed.

# 00:00 / Person overboard. Launch the fleet.

7 minutes · MAYDAY · Organize the fleet around the last report.

## 00:00 — A person overboard. A fleet of functions.

Someone falls overboard. We have a last report, several boats, currents, and incomplete information. We need to organize, decide, learn, and eventually explain what each boat contributed. I want to rebuild those pieces with you. The star of this talk is not a particular optimizer, and it is not a tour of Clojure syntax. It is a small vocabulary of higher-order functions. For the Clojure, Haskell, and Elixir developers here, the mechanics will be familiar. Watch what happens when we use that familiarity to read the mathematics. For the Lean developers: later, we will separate something that runs from something we have proved.

> Establish the rescue story. Point to the last-report marker. Explain that this is a schematic teaching model, not an operational rescue system.

```
objective → gradient → step

(iterate step initial-state)
```

## 01:40 — A mini-language makes ordinary data.

We could type a long vector of numbers. Instead, we can make a tiny language: a grid of boats. Originally I used birds; changing the picture does not change the mathematics. The macro translates that literal grid into coordinates. It does not invent a new runtime or a special kind of optimizer. After expansion, this is ordinary immutable data. That is one Clojure advantage worth showing: a domain notation can disappear into a representation that all our existing functions understand. We should use the notation to make the problem easier to see, then get out of its way.

> Open Clojure briefly and show the literal boat grid. Return to stage view. Point to the expanded vector and ask which representation the optimizer actually receives; the answer is coordinates, not boat symbols.

```
(f/fleet [⛵ · · ⛵]
         [· · ⛵ ·])

;; => [0.0 0.0 3.0 0.0 2.0 1.0]
```

## 03:10 — An objective configures a step.

The objective takes the whole formation and returns one number: the sum of squared distances to the report. Numerical differentiation takes that function and returns another function, one that gives a direction at a formation. Descent-step captures that gradient function and a rate. What comes back is just State to State. The loop is now supplied by iterate. In Clojure, iterate includes the initial state, so 101 states means 100 updates. We can take a prefix, keep a history, inspect it, or visualize it without rewriting the algorithm. These are optimization iterations over candidate formations; they are not physical boat movements.

> Show the first update, then play. Open One update and identify position, gradient, and base movement. Unfold HOF if time permits.

```
(def step
  (h/descent-step
    (h/gradient objective) 0.025))

(take 101 (iterate step {:x boats}))
```

## 05:10 — A bad objective, correctly optimized.

Here is the first useful failure. All the boats end up on top of each other. This is not a mysterious numerical problem. We rewarded exactly that behavior. Our objective said nothing about separation. Before blaming an optimizer, inspect the meaning of the function it is following. Now we have a concrete next requirement: boats should stay apart. Do we need a new optimization loop? Or can we change the function that the existing loop already understands?

> Jump to the last formation. Pause. Let the audience explain the crowding before advancing.

```
;; Minimize only distance:
(sum-of-squared-distances boats report)

;; The best answer puts every boat there.
```

**Transition:** The optimizer did exactly what we asked. We forgot to ask the boats to stay apart.

**Recovery:** Use recorded playback. The point is the shape of the step, not a live typing performance.

**Cut if behind:** Skip unfolding numerical differentiation. Keep the macro, returned function, and iterate.

# 07:00 / Change the landscape.

3 minutes · KEEP CLEAR · Make crowding expensive.

## 07:00 — A constraint is another function.

For each pair of boats, define a function of the formation. It returns 1.1 minus their separation. A positive result means a violation. The representation is deliberately unremarkable: another function from the same coordinate vector to a number. We can collect these functions, map over them, and reduce their costs. We have described the new requirement without saying anything about iteration.

> Point to separation halos and the closest-pair distance.

```
g(x) = 1.1 − distance(boatᵢ, boatⱼ)

violation = max(0, g(x))
```

## 07:55 — Wrap the objective, keep the solver.

Penalize accepts the original objective, a sequence of constraint functions, and a weight. It returns a new objective. The returned function adds the original cost to three times the sum of squared positive violations. The descent code is unchanged because its interface is unchanged. This is the useful meaning of composition here: we can alter the problem at the exact boundary where the algorithm accepts a function.

> Unfold penalize. Read the returned fn first, then map and reduce inside it.

```
(def safer-objective
  (h/penalize objective constraints 3.0))

;; Objective → Objective
```

## 08:55 — A soft penalty is a tradeoff.

The boats separate, but do not read this picture as a collision guarantee. With a finite penalty weight, the cheapest compromise can still violate the requested distance. The inspector makes that visible: there is a distance cost, a crowding cost, and a closest-pair

distance. Later, Rainbow will give us a related but different result: projection into ordered bounds has a proved invariant. A soft penalty and a projection are distinct wrappers with distinct claims.

> Advance to the end and compare total, distance, penalty, and closest distance.

```
total cost = distance cost + penalty

;; inspect both components
```

**Transition:** We changed what a good answer means. Next, change how a step moves.

**Recovery:** The two cost components in One update show the compromise even if the motion is hard to see.

**Cut if behind:** Keep the wrapper signature and soft-penalty caveat; skip pair enumeration.

# 10:00 / Give a function momentum.

4 minutes · CARRY MOTION · Change a step without changing iterate.

## 10:00 — Ask the base step for a proposal.

Now I want to modify the algorithm rather than the objective. A momentum step combines the new proposal with some of the previous movement. The wrapper does not need to know how the base step computed its displacement. It calls the supplied step and subtracts the old coordinates. That is a very small contract, and it is enough to build the variation. Notice where the abstraction boundary sits: around a step, not around a giant optimizer object.

> Press T to show the zero-carry first update as large numbers. Step once to reveal carried velocity. Return to Explanation before the next cue.

```
proposal = step(state)
delta    = proposal.x − state.x
```

## 11:10 — History is a value in the state.

The history we need is just the previous velocity. Put it in the state map. There is no hidden accumulator and no special requirement on iterate. Call this step twice with the same input state and you get the same output. On the first update the old velocity is zero; on the next, you can see the carried component. Immutable state has not removed history. It has made history explicit, inspectable, and replayable.

> Use One update at frames 0 and 1. Add base movement to carried movement and compare the next position.

```
v′ = 0.75 × v + delta
x′ = x + v′

{:x x′ :velocity v′}
```

## 12:30 — Return the same shape of function.

The wrapper accepts State to State and returns State to State. Iterate remains completely indifferent. On the local JVM, try beta zero: it removes the carried displacement and recovers the base position trajectory. Then try a larger beta and watch

overshoot. Momentum is not a universal promise of speed; it is a particular recurrence with assumptions that matter. The reusable insight is that algorithm variations can be expressed as transformations of the step function.

> Unfold the wrapper. At frame 10, use Change one thing to compare beta zero and beta 0.9. Press T for the calculation; the frame stays fixed. Choose Talk default before returning to the script. Locally, the same change can be typed into Clojure.

```
(def step
  (h/with-momentum plain-step 0.75))

(iterate step initial-state)
```

**Transition:** We have changed the objective and the step. We can also supply how independent work is mapped.

**Recovery:** Use steps 0 and 1 in One update; the carry is zero first and nonzero next.

**Cut if behind:** Skip the optional beta experiment. Keep the explicit state and the proposal-plus-carry arithmetic.

# 14:00 / Even execution is an argument.

2 minutes · EVALUATE OPTIONS · Score independent formations.

## 14:00 — The mapping strategy is an argument.

We have 32 candidate formations. Each fitness calculation runs a small trajectory and scores it. One candidate does not need the answer from another candidate. That is the boundary we can parallelize. Supply map or pmap to the same evaluator; keep the objective and inputs identical. We are not parallelizing dependent steps within one trajectory. We are mapping independent trajectories.

> Press T to show the actual map/pmap timings and equality check. The visible fleet supplies workload context, not thread activity. Return to Explanation for the next cue.

```
(evaluate-population map  fitness population)
(evaluate-population pmap fitness population)
```

## 15:00 — Force the work before timing it.

Both mapping strategies can defer work. Vec realizes the results before timing stops. We compare the ordered scores, not just two stopwatch numbers. This is a single local measurement, with JVM and coordination overhead included. If pmap is slower, the lesson survives. Higher-order functions expose a clean execution choice; they do not repeal the cost of scheduling work.

> State whether this measured run was faster or slower without treating it as a benchmark claim.

```
(vec (map-fn score population))

(= serial-results parallel-results)
```

**Transition:** So far the state has been coordinates. What if the state being improved were a function?

**Recovery:** A slower pmap is a valid outcome. Explain coordination overhead; never rerun until a flattering number appears.

**Cut if behind:** One signature, one equality check, then move on.

# 16:00 / The next state is a function.

6 minutes · ROUTE TO RESCUE · First assume a confirmed location.

## 16:00 — A map can be a value function.

For a moment, assume the person's location has been confirmed. Gold is the destination, dark blocks are reefs, and red is a hazardous shoal. A move may be pushed sideways by the current. We want a good action from each boat location. A value function estimates future return from a location. In this finite example we can materialize the function as a map, and a Clojure map is callable. Our next algorithmic state is therefore a function, represented by data.

> Identify the destination, shoal, reefs, heights, and policy arrows.

```
V : Location → ExpectedReturn

;; Clojure maps are callable.
(V [5 0])
```

## 17:25 — One action is a weighted reduction.

Consider the outlined square beside the destination. An action can have three outcomes because of the current. For each outcome, add the immediate reward to 0.92 times the old value of the next state. Multiply by its probability, then sum. That intimidating expectation is a map followed by a reduce. The old value function is an input to the calculation. The inspector exposes all three terms, so we can account for the exact number assigned to the action.

> Open One update. Read one outcome row and sum the final column.

```
Q(s,a) = Σ p(s′|s,a) ×
             [reward + 0.92 × V(s′)]
```

## 19:00 — Bellman returns a new value function.

For each state, score the available actions and choose the maximum. The outer function receives V. The inner function answers a question about a state. That nested pair of functions is the Bellman operator made visible in code. All states in a backup read the

same old V; we materialize the resulting function before the next iteration. We avoid accidentally mixing old and new values by making the function boundary explicit. Terminal values are zero because the rescue or hazard reward was already paid on entry.

> Show the nested fn forms in Clojure, then step from V0 to V1.

```
(fn [value]
  (fn [state]
    (apply max
      (map #(lookahead value state %)
           (actions state)))))
```

## 20:35 — The same iterate, a different state.

As we iterate, reward information propagates through possible futures. The height of a square is a value estimate, and the arrow is a greedy action. These heights are not probabilities. Earlier we iterated a transformation of coordinates. Here we iterate a transformation of functions. The common shape is real, but it does not mean all the mathematics is the same. Bellman's contraction result, which we will revisit with Rainbow, is a reason this particular operator behaves well under its assumptions.

> Play several backups, then pause at frame 12. If time permits, select Discount gamma = 0 and watch distant value disappear without changing the operator interface. Choose Talk default before leaving the chapter.

```
(iterate improve initial-value)

;; (State → State)
;; where State is a value function
```

**Transition:** Now remove the assumption that we know where the person is. The state changes, but the operator's shape survives.

**Recovery:** Inspect cell [5,0] at frame 0. The action table and weighted outcomes carry the explanation.

**Cut if behind:** Skip late convergence frames. Keep one weighted backup and the function-to-function signature.

# 22:00 / The person is drifting. Our belief moves.

7 minutes · SEARCH UNDER UNCERTAINTY · A miss changes the belief.

## 22:00 — We do not know the location.

The person is drifting and we cannot see the actual location. We therefore carry a probability distribution over possible locations. Do not mistake a bright square for a picture of the person. It represents our uncertainty. This small model uses exact rational probabilities. Current moves the person east, south, or keeps them in place; blocked movement stays. A search covers a sector, with 80 percent detection inside and no false positives. We abstract away travel time here so that we can focus on the information update.

> Point to the initial probability mass, then toggle Predicted and Posterior after the first observation.

```
belief : Location → Probability
```

```
{[3 0] 1/2, [4 0] 1/3, [4 1] 1/6}
```

## 23:30 — Prediction transports probability mass.

First advance the world model. Every old location contributes mass to its possible successors. Weight each transition by how likely the old location was, then add contributions that arrive at the same place. This is a reduction over distributions. The drift function is supplied to the belief updater. We can inspect prediction separately from observation, which is important: drift changes the distribution even before a boat reports anything.

> At the first miss, choose Predicted. Compare the shape with the prior, without claiming the person actually followed any particular path.

```
predicted(s′) = Σ belief(s) × T(s,s′)
```

```
(predict drift belief)
```

## 25:05 — A miss is evidence, not elimination.

Now a boat searches and reports no sighting. Inside its sector, a miss was possible but less likely: probability one fifth. Outside the sector, a miss was certain. Multiply by those likelihoods, sum to get the probability of the observation, and divide to normalize. A missed sector loses mass but is not eliminated. Reductions makes the whole history available: the initial belief, the posterior after the first miss, and the next. The sensor and transition functions configure the reducer. This is decision-making infrastructure built from functions accepting functions.

> Choose Posterior and use One update to compare predicted mass, likelihood, and posterior. Advance to the second miss.

```
posterior ∝ likelihood × predicted

(reductions (belief-step drift sensor)
            prior observations)
```

## 26:55 — Bellman now acts on beliefs.

We can ask which sector to search first if we get two searches. An immediate sighting succeeds. A miss leads to a new belief, and from that belief we choose the second action. The value function's argument has changed from a location to a probability distribution. Bellman still accepts a value function and returns another. Our two-search calculation is exact for this tiny model, with no discount or travel cost. The animation replays scripted observations; it is not pretending to be an optimal policy rollout. Distinguishing those two objects makes the example honest and easier to understand.

> At frame 2 press Plan two searches. Compare all three first-action scores, then read the winning seen/miss decomposition. The second action is chosen from the miss posterior after another drift step. Open the notebook for every branch. Show the final sighting, then return to frame 2 before attribution.

```
Q(b,a) = P(seen)
       + P(miss) × V(b′miss)

(iterate belief-bellman (constantly 0))
```

**Transition:** We can now value the fleet's search coverage. But overlapping boats should not each claim the same gain.

**Recovery:** Step the four recorded beliefs. The scripted observations and two-search planning table are distinct.

**Cut if behind:** Keep Bayes and the belief-Bellman signature. Skip derivation of the two-search scores.

# 29:00 / Who made the difference?

4 minutes · SHARE THE CREDIT · Attribute expected search coverage.

## 29:00 — A coalition gets one shared belief.

We now send three search boats into overlapping sectors. The value of a coalition is its expected detection coverage under a fixed belief. We reuse the posterior after two misses from the previous example, then predict the next drift. Every coalition is compared against that same distribution. That shared reference matters: otherwise we would be attributing differences in assumptions rather than differences in contribution. This is prospective expected coverage, not a moral judgment about who happened to make the final sighting.

> Show the sector coverage map. Identify the shared-belief connection to the previous chapter.

```
(def value-of
  (rescue-value predicted-belief))

;; Coalition → Detection coverage (pp)
```

## 30:15 — Overlap changes marginal contribution.

If two boats can detect the person independently at the same location, their combined detection probability is one minus the chance they both miss: 96 percent. They cannot both claim an additional 80 percentage points. Now let boats arrive in an order. Each gets the difference between the coalition value after and before it arrives. The fold carries the coalition and its ledger. Reductions lets us watch the marginal contribution happen, instead of just displaying a final allocation.

> Step the first three arrivals. Point to the before, after, and gain values.

```
1 − (1 − 0.8)² = 0.96

gain = value(team ∪ {boat}) − value(team)
```

## 31:35 — Average over orders, not over subsets.

An early boat gets credit for coverage that a later boat would otherwise supply. So average over every arrival order. With three boats, there are only six. The result is a Shapley allocation. The bottom bars are the final average, while the animated coalition shows one particular order. The inspector shows the six actual gains behind each bar. This enumeration has factorial cost; it is for understanding the definition and for tiny games. The reusable machinery receives value-of as a function. Change the game, keep the fold and the averaging.

> Compare the first and last orders. Open Clojure, then the One update notebook tab to show all six marginal gains and their means. Show that credits sum to the fleet's total expected coverage. Press S to return to stage.

```
(map #(arrival-trace value-of %)
     (permutations boats))

;; Average each boat's marginal gains.
```

**Transition:** Now we have computations and properties we want. Which properties can our theorem prover actually check?

**Recovery:** Use one complete arrival order and the six recorded marginal gains.

**Cut if behind:** Skip a second permutation. Keep the overlap example and averaging argument.

# 33:00 / From a function to a claim we can check.

4 minutes · CHECK THE CLAIM · Separate execution from proof.

## 33:00 — A theorem statement is not a proof.

This is Rainbow, my theorem prover. It represents statements and proof terms as Clojure data, and a kernel checks whether a proof term has the claimed type. For this talk we replayed an existing checkpoint and rechecked 11 selected declarations. Their axiom closures are empty. That is stronger evidence than an animation behaving plausibly, but we must still say exactly what was proved. A statement about exact mathematical terms is not automatically a proof that my floating-point renderer or optimizer implements those terms correctly.

> Open Clojure, then the One update notebook tab. Expand a declaration to show its theorem identifier and empty axiom closure; the audit timestamp is below the list. Press S to return to stage. The large T calculation card does not contain these metadata.

```
(kernel/check env
  (:rainbow/value declaration)
  (:rainbow/type declaration)
  (set (:rainbow/universes declaration)))
```

## 34:05 — Momentum has an exact recurrence.

For momentum, the checked development has signed scalar position and velocity, nonnegative rational rate, beta, and gradient output. It proves the recurrence and this worked transition. It does not prove that momentum always accelerates convergence. For projection, a related checked theorem says that an exact rational vector step clamped to ordered bounds stays inside them. It does not turn our soft separation penalty into a safety proof. These are different claims, and keeping the boundaries visible is part of the demonstration.

> Show the worked recurrence, then briefly the projection invariant.

```
v′ = 1 × 2 − 1 × 1 = 1
x′ = 0 + 1 = 1
```

## 35:10 — Bellman has a bound on change.

The strongest direct connection is Bellman. Under the theorem's assumptions, value functions within epsilon yield backups within gamma epsilon. The transitions must be nonnegative and normalized, with admissible actions. If gamma is less than one, this becomes a strict contraction. This particular checked theorem is a bound; do not quietly add a claim of fixed-point existence to it. Rainbow has additional uniqueness and conditional convergence developments, but they are separate results. The new rescue belief planner and our general permutation Shapley implementation are not proved by this audit.

> The cue selects the contraction frame. Read its bound on stage; if showing the exact theorem identifier, open the One update notebook tab and expand that declaration, then return with S.

```
|V(s) − W(s)| ≤ ε
              ↓ Bellman
|B(V)(s) − B(W)(s)| ≤ γ ε
```

## 36:20 — Try a claim without a proof.

Finally, replace the momentum proof with the natural number zero. The kernel rejects it. A familiar-looking theorem name or an appealing example does not grant acceptance. The artifact must actually check. That is the distinction I want to bring back to ordinary programming: functions give us clear interfaces for composition, and those interfaces also give us concrete places to state properties precisely.

> Show the rejected result. Explicitly call it recorded kernel evidence, not a kernel running in the browser.

```
0 : MomentumRecurrence ?

;; kernel result: rejected
```

**Transition:** What ties this together is the vocabulary of functions, along with the properties we can state about them.

**Recovery:** Use the recorded kernel evidence. The recheck command is clojure -M:rainbow; do not wait for a full audit on stage.

**Cut if behind:** Keep momentum, Bellman, and one rejection. Mention projection briefly or omit it.

# 37:00 / One rescue. One vocabulary.

3 minutes · THE WHOLE MISSION · Different state spaces, the same functional vocabulary.

## 37:00 — Different state spaces. Familiar shapes.

Pull back from the details. A candidate formation is a state. A value function can also be a state. So can a probability distribution, or a coalition with a contribution ledger. Iterate repeatedly applies a transformation. Reductions exposes a fold's intermediate states. Map expresses independent applications. The rescue connected the questions, but these operators organized the answers. The mathematical assumptions did not disappear; the functional structure gave us a place to put them.

> Let the camera pull back. Trace the connections between the three areas of the chart.

```
iterate    : (A → A) → A → [A]
reductions : (S × X → S) → S → [X] → [S]
map        : (A → B) → [A] → [B]
```

## 38:15 — Functions are the architecture.

A penalty transformed an objective. Momentum transformed a step. Bellman transformed a value function. A sensor and transition model configured a belief reducer. A coalition-value function configured attribution. This is what I mean by rebuilding operations research in higher-order functions: not pretending every algorithm is identical, but exposing where the interesting choices enter and how they compose. It lets us change behavior without rebuilding the entire control structure.

> Pause on the transformation signatures. Leave a beat of silence before the closing invitation.

```
Objective → Objective
Step → Step
ValueFunction → ValueFunction
```

## 39:15 — Keep exploring the functions.

These ideas are developed further in my book, Applied Higher Order Functions. Rainbow explores the next question: which properties of those constructions can we express and check? I hope you leave with a way to approach an intimidating algorithm. Find its state. Find its step. Find the functions that configure that step. Then make those boundaries visible enough to inspect, experiment with, and perhaps prove something about. Thank you.

> Keep the closing card visible for questions. Do not navigate back into code while thanking the audience.

```
Read the function.
Change the function.
Inspect the consequence.
State the property.
```

**Transition:** Thank the audience and invite questions.

**Recovery:** The closing card contains all three HOF signatures and the book and Rainbow names.

**Cut if behind:** End with the three signatures and the central claim. Reserve questions for after the 40-minute slot.